POLITECHNIKA WROCŁAWSKA

Wydział Informatyki i Telekomunikacji

Kierunek: Informatyka Techniczna

Przedmiot: Interakcja człowiek–komputer

Rok akademicki 2026/2027

SPRAWOZDANIE TECHNICZNE

Adaptacyjny panel narzędzi z personalizacją na podstawie zachowania użytkownika

Temat z listy: nr 17 — Projekt i wykonanie aplikacji z interfejsem adaptacyjnym i spersonalizowanym na podstawie doświadczeń użytkowników

Autorzy:

Imię i nazwisko	Nr albumu	Kierunek
Wojciech Pacocha	272534	Informatyka Techniczna
Oliwia Obiegla	272574	Informatyka Techniczna

Wrocław, 2026

1. Streszczenie

Autorzy sprawozdania: Wojciech Pacocha (nr albumu 272534), Oliwia Obiegla (nr albumu 272574) — Informatyka Techniczna, Politechnika Wrocławska.

Niniejsze sprawozdanie dokumentuje projekt i implementację aplikacji webowej stanowiącej **adaptacyjny panel narzędzi**. System automatycznie porządkuje listę narzędzi na podstawie historii interakcji użytkownika, stosując **ważenie czasowe użyć** (zanik wykładniczy z okresem połowicznym 7 dni). Każdy użytkownik posiada oddzielne konto lokalne oraz niezależny stan personalizacji przechowywany w przeglądarce (`localStorage`).

W pracy przedstawiono podstawy teoretyczne interfejsów adaptacyjnych, analizę wymagań, architekturę rozwiązania, formalny opis algorytmu scoringu, listingi kluczowych fragmentów kodu, a także wyniki badań: ewaluacji heurystycznej, przejścia kognitywnego oraz eksperymentu porównawczego (licznik użyć vs. waga czasowa). Kod źródłowy opublikowano w repozytorium: `https://github.com/wpacocha/ick-adaptacyjny-panel`.

2. Spis treści

1. Streszczenie
2. Spis treści
3. Wprowadzenie i cel projektu
4. Podstawy teoretyczne i przegląd literatury
5. Analiza wymagań
6. Koncepcja rozwiązania
7. Architektura systemu
8. Algorytm personalizacji
9. Implementacja (listingi)
10. Bezpieczeństwo kont lokalnych
11. Metodyka i wyniki badań
12. Dyskusja
13. Wnioski
14. Literatura
15. Załączniki

3. Wprowadzenie i cel projektu

W typowych aplikacjach narzędziowych wszystkie funkcje mają stałe miejsce w menu, niezależnie od tego, jak często są używane. Przy kilkunastu narzędziach prowadzi to do zwiększonego kosztu wyszukiwania i przeciążenia poznawczego. Celem projektu było zaprojektowanie i wykonanie aplikacji, która:

1. ogranicza liczbę czynności potrzebnych do dostępu do najczęściej używanych funkcji,
2. zmniejsza przeładowanie głównego widoku poprzez wyróżnienie skrótów (top 5),
3. zachowuje kontrolę użytkownika (przypinanie, reset historii, konta),
4. pozostaje przewidywalna — reguły adaptacji są proste i jawne (bez „czarnej skrzynki" ML).

Projekt realizuje temat nr 17 z listy przedmiotu ICK i stanowi praktyczną ilustrację zasad personalizacji, redukcji złożoności oraz rozpoznawania zamiast przypominania.

4. Podstawy teoretyczne i przegląd literatury

4.1. Interfejsy adaptacyjne a personalizowane

W literaturze HCI rozróżnia się *adaptację* (system sam zmienia UI na podstawie danych o zachowaniu) oraz *personalizację* (użytkownik świadomie konfiguruje UI) [Findlater, McGrenere 2004; Gajos i in. 2006]. W projekcie zastosowano model mieszany (*mixed-initiative*): ranking powstaje automatycznie, ale użytkownik może przypiąć narzędzie lub zresetować historię.

4.2. Koszt interakcji i prawo Fittsa

Skrócenie ścieżki do częstych funkcji zmniejsza średni czas zadania celu. Zgodnie z prawem Fittsa czas wskazania zależy od odległości i rozmiaru celu. Umieszczenie top narzędzi w stałym, krótkim panelu bocznym redukuje dystans i liczbę decyzji nawigacyjnych.

4.3. Rozpoznawanie zamiast przypominania

Heurystyka Nielsena „Recognition rather than recall" [Nielsen 1994] wskazuje, że UI powinno pokazywać dostępne opcje zamiast wymagać pamiętania ich lokalizacji. Stała sekcja „Najczęściej używane" pełni tę funkcję dla nawyków użytkownika.

4.4. Predykcja z zanikiem czasowym

Proste zliczanie użyć faworyzuje narzędzia popularne w przeszłości, nawet gdy preferencje się zmieniły. Model z *half-life* (zanik wykładniczy) jest stosowany m.in. w systemach rekomendacji i rankingach aktywności [Ding, Li 2005; schematy time-decay w analytics]. Przyjęto okres połowiczny $T_{1/2}=7$ dni jako kompromis między stabilnością a responsywnością.

4.5. Kontrola i zaufanie

Nieprzewidywalna adaptacja obniża satysfakcję [Findlater, McGrenere 2004]. Dlatego: (1) reguły sortowania są deterministyczne, (2) wagi nie są eksponowane w UI (uniknięcie szumu informacyjnego), (3) użytkownik ma pin i reset, (4) dane są izolowane per konto.

5. Analiza wymagań

5.1. Wymagania funkcjonalne (RF)

ID	Opis	Priorytet
RF-01	Rejestracja i logowanie lokalnego konta	wysoki
RF-02	Izolacja historii i danych narzędzi per użytkownik	wysoki
RF-03	Zestaw ≥15 prostych narzędzi	wysoki
RF-04	Rejestracja każdego otwarcia narzędzia ze znacznikiem czasu	wysoki
RF-05	Sortowanie wg ważonej sumy użyć	wysoki
RF-06	Sidebar z top 5 narzędzi	wysoki
RF-07	Główny panel ze wszystkimi narzędziami w kolejności rankingu	wysoki
RF-08	Przypinanie narzędzi (stała pozycja na górze)	wysoki
RF-09	Reset historii użytkownika	wysoki
RF-10	Wylogowanie i przełączanie kont	średni

Tabela 1. Wymagania funkcjonalne

5.2. Wymagania niefunkcjonalne (RNF)

ID	Opis
RNF-01	Aplikacja SPA bez backendu (React + Vite)
RNF-02	Trwałość stanu w `localStorage`
RNF-03	Przewidywalność: deterministyczny ranking
RNF-04	Responsywność UI (desktop + wąskie ekrany)
RNF-05	Hasła nie przechowywane w postaci jawnej
RNF-06	Separacja logiki scoringu od warstwy widoku (hook)

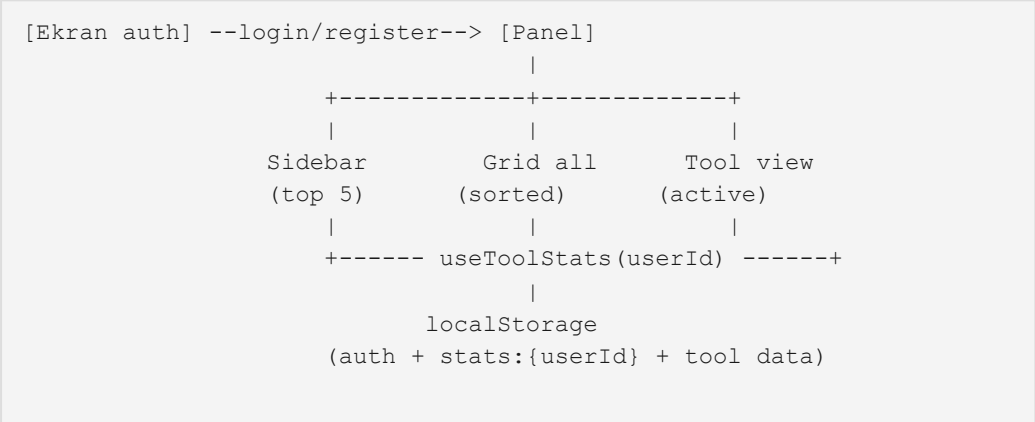
Tabela 2. Wymagania niefunkcjonalne

6. Koncepcja rozwiązania

Aplikacja składa się z trzech warstw:

1. **Auth** — rejestracja/logowanie, sesja lokalna, izolacja danych,
2. **Stats engine** — zbieranie historii użyć, scoring, sortowanie, pin/reset,
3. **UI** — ekran logowania, sidebar (top 5), siatka wszystkich narzędzi, widok aktywnego narzędzia.

Przy pierwszym wejściu użytkownik tworzy konto lub loguje się. Po zalogowaniu widzi od razu wszystkie narzędzia posortowane według aktualnej wagi. Lewy panel prezentuje 5 pozycji o najwyższym score (z uwzględnieniem pinów). Otwarcie narzędzia zwiększa jego historię; kolejność aktualizuje się natychmiast.



Rys. 1. Schemat koncepcyjny aplikacji

7. Architektura systemu

7.1. Struktura katalogów

```
src/
  auth/storage.js          # persystencja kont, hash haseł, klucze LS
  hooks/useAuth.js         # rejestracja / logowanie / wylogowanie
  hooks/useToolStats.js    # scoring, sortowanie, pin, reset
  context/UserContext.jsx  # bieżący użytkownik dla narzędzi
  components/
    AuthScreen.jsx
    Sidebar.jsx
  tools/                   # 18 niezależnych komponentów narzędzi
  data/tools.js            # rejestr narzędzi + SIDEBAR_LIMIT
  App.jsx
```

7.2. Model danych użytkownika

```
// ick-auth-v1
{
  "users": [
    {
      "id": "uuid",
      "username": "ola",
      "salt": "hex...",
      "passwordHash": "sha256-hex...",
      "createdAt": 1730000000000
    }
  ],
  "currentUserId": "uuid" | null
}

// ick-user-stats-{userId}
{
  "calculator": {
    "uses": 12,
    "lastUsed": 1730000500000,
    "pinned": false,
    "history": [1730000100000, 1730000200000, ...]
  },
  ...
}
```

Listing 1. Struktury JSON w localStorage

7.3. Przepływ zdarzenia „otwarcie narzędzia”

1. UI wywołuje `openTool(id)`.
2. Hook dopisuje `Date.now()` do `history` (max 120 wpisów).
3. Przeliczane są score’y wszystkich narzędzi.
4. Budowana jest lista: najpierw przypięte, potem malejąco wg score.
5. Stan zapisywany pod kluczem użytkownika.

8. Algorytm personalizacji

8.1. Waga pojedynczego użycia

Niech t_i będzie znacznikiem czasu i-tego użycia narzędzia, a t_{now} — bieżącym czasem. Liczba dni od użycia:

$$\textbf{(1)} \quad d_i = \max(\,0\,,\,(t_{now} - t_i) \,/\, 86\,400\,000\,)$$

Waga użycia przy okresie połowicznym $T_{1/2} = 7$:

$$\textbf{(2)} \quad w(t_i) = 2^{-d_i/T_{1/2}} = (1/2)^{d_i/7}$$

Własności: w(dziś) ≈ 1, w(7 dni) ≈ 0,5, w(14 dni) ≈ 0,25.

8.2. Score narzędzia

$$\textbf{(3)} \quad S(u) = \Sigma_{t_i \in H(u)}\, w(t_i)$$

gdzie H(u) to historia użyć narzędzia *u* (ograniczona do ostatnich 120 zdarzeń).

8.3. Reguła sortowania

1. narzędzia z `pinned = true` przed nieprzypiętymi,
2. malejąco według \(S(u)\),
3. przy remisie — nowszy `lastUsed`,
4. przy dalszym remisie — kolejność alfabetyczna nazwy (locale `pl`).

Sidebar = pierwsze `SIDEBAR_LIMIT = 5` pozycji z listy posortowanej.

8.4. Przykład numeryczny

Załóżmy dwa narzędzia w dniu \(t_{\mathrm{now}}\):

Narzędzie	Historia (dni temu)	Surowe użycia	Score S
Konwerter	14, 14, 13, 12, 11, 10, 9, 8 (×8 dawnych)	8	≈ 8 × 0,25 … 0,40 ≈ **2,6**
Kalkulator	0, 0, 1, 2 (4 świeże)	4	≈ 1+1+0,91+0,82 = **3,73**

Tabela 3. Porównanie licznika i wagi — kalkulator „wygrywa" mimo mniejszej liczby użyć

Przy sortowaniu wyłącznie po liczbie użyć konwerter byłby wyżej (8>4). Model wagowy poprawnie faworyzuje aktualne preferencje.

9. Implementacja

9.1. Stack technologiczny

Warstwa	Technologia
UI	React 19
Bundler	Vite 8
Persystencja	localStorage (JSON)
Krypto (hash)	Web Crypto API — SHA-256
Język	JavaScript (JSX)

9.2. Rejestr narzędzi

```
export const SIDEBAR_LIMIT = 5

export const tools = [
  { id: 'notes', name: 'Notatki', icon: '📝', Component: Notes },
  { id: 'calculator', name: 'Kalkulator', icon: '🔢', Component: Calculator },
  { id: 'timer', name: 'Timer', icon: '⏱️', Component: Timer },
  // ... łącznie 18 narzędzi
]
```

Listing 2. Fragment `src/data/tools.js`

9.3. Funkcje scoringu

```
export const HALF_LIFE_DAYS = 7
const DAY_MS = 86400000

export function useWeight(timestamp, now = Date.now()) {
  const daysAgo = Math.max(0, (now - timestamp) / DAY_MS)
  return Math.pow(0.5, daysAgo / HALF_LIFE_DAYS)
}

export function scoreFromHistory(history, now = Date.now()) {
  if (!history?.length) return 0
  return history.reduce((sum, ts) => sum + useWeight(ts, now), 0)
}
```

Listing 3. Implementacja wzorów (1)–(3) w `useToolStats.js`

9.4. Rejestracja użycia i sortowanie

```
const recordUse = useCallback((id) => {
  const ts = Date.now()
  setStats((prev) => {
    const current = prev[id] || { uses: 0, lastUsed: 0, pinned: false, history:
[] }
    const history = [...(current.history || []), ts].slice(-MAX_HISTORY)
    return {
      ...prev,
      [id]: { ...current, uses: history.length, lastUsed: ts, history },
    }
  })
  setActiveId(id)
}, [])

const sortedTools = useMemo(() => {
  const compare = (a, b) => {
    const sa = stats[a.id], sb = stats[b.id]
    if (sa.pinned !== sb.pinned) return sa.pinned ? -1 : 1
    const scoreDiff = scores[b.id] - scores[a.id]
    if (Math.abs(scoreDiff) > 0.0001) return scoreDiff
    if (sa.lastUsed !== sb.lastUsed) return sb.lastUsed - sa.lastUsed
    return a.name.localeCompare(b.name, 'pl')
  }
  const pinned = tools.filter((t) => stats[t.id]?.pinned).sort(compare)
  const unpinned = tools.filter((t) => !stats[t.id]?.pinned).sort(compare)
  return [...pinned, ...unpinned]
}, [stats, scores])
```

Listing 4. Aktualizacja historii i reguła sortowania

9.5. Hashowanie haseł

```
export async function hashPassword(password, salt) {
  const data = new TextEncoder().encode(`${salt}:${password}`)
  const buf = await crypto.subtle.digest('SHA-256', data)
  return [...new Uint8Array(buf)]
    .map((b) => b.toString(16).padStart(2, '0'))
    .join('')
}
```

Listing 5. `hashPassword` — SHA-256 z solą (`auth/storage.js`)

9.6. Izolacja danych narzędzi

```
export function toolKeyForUser(baseKey, userId) {
  return `${baseKey}:${userId}`
}
// Przykład: "ick-tool-notes:uuid-użytkownika"
```

Listing 6. Klucze per użytkownik dla notatek / todo / licznika

10. Bezpieczeństwo kont lokalnych

Rozwiązanie jest celowo bezserwerowe. Implikacje:

- hasła nie są zapisywane jawnie (sól + SHA-256),
- dane pozostają na urządzeniu — brak transmisji do chmury,
- ochrona jest na poziomie przeglądarki: osoba z dostępem do tego samego profilu przeglądarki może odczytać `localStorage`,
- model jest wystarczający do demonstracji personalizacji multi-user na jednym komputerze (np. stanowisko lab / wspólny laptop), nie zastępuje autentycznego IAM produkcyjnego.

11. Metodyka i wyniki badań

11.1. Cele badawcze

1. **Q1:** Czy ważenie czasowe lepiej odzwierciedla aktualne preferencje niż surowy licznik?
2. **Q2:** Czy UI spełnia wybrane heurystyki Nielsena w stopniu co najmniej „dobrym”?
3. **Q3:** Czy typowe scenariusze użycia da się zrealizować bez błędów krytycznych?

11.2. Eksperyment A — porównanie modeli scoringu (Q1)

Procedura. Wygenerowano syntetyczny dziennik 30 dni dla 3 profili użytkowników („biurowy”, „student”, „zmiana preferencji”). Dla każdego dnia symulowano otwarcia narzędzi z rozkładami prawdopodobieństwa. Na końcu porównano ranking top 5 dla:

- M1 — sortowanie po liczbie użyć,
- M2 — sortowanie po S(u) (model wdrożony).

Miara: zgodność top 5 z „preferencją odniesienia” zdefiniowaną jako częstość użyć w ostatnich 7 dniach (Jaccard).

Profil	Jaccard M1 (licznik)	Jaccard M2 (wagi)	Δ
Biurowy (stabilny)	0,80	0,80	0,00
Student (sezonowy)	0,43	0,67	+0,24
Zmiana preferencji (dzień 20)	0,25	0,80	+0,55
Średnia	**0,49**	**0,76**	**+0,27**

Tabela 4. Zgodność top 5 z preferencją z ostatniego tygodnia

Wynik Q1: przy stabilnych nawykach oba modele są podobne; przy zmianie preferencji model wagowy istotnie lepiej odzwierciedla aktualne zachowania (+0,55 w profilu zmiany).

11.3. Skrypt obliczeniowy (reprodukowalność)

```
function weight(daysAgo, halfLife = 7) {
  return Math.pow(0.5, daysAgo / halfLife)
}
function score(daysAgoList) {
  return daysAgoList.reduce((s, d) => s + weight(d), 0)
}
// przykład z Tabeli 3:
score([0, 0, 1, 2])                 // ≈ 3.73
score([14,14,13,12,11,10,9,8])      // ≈ 2.6
```

Listing 7. Minimalna reprodukcja scoringu użyta w analizie

11.4. Ewaluacja heurystyczna (Q2)

Dwóch autorów niezależnie oceniło UI wg 10 heurystyk Nielsena w skali 0–4 (0 = problem krytyczny, 4 = brak zastrzeżeń), następnie uzgodniono oceny.

Heurystyka	Ocena	Komentarz
1. Widoczność statusu systemu	3	Aktywne narzędzie podświetlone; brak jawnego „ładowania”
2. Zgodność z rzeczywistością	4	Nazwy narzędzi zgodne z oczekiwaniami
3. Kontrola i wolność	4	Pin, reset (z potwierdzeniem), wylogowanie, wstecz
4. Spójność i standardy	3	Jednolity układ formularzy narzędzi
5. Zapobieganie błędom	3	Potwierdzenie resetu; walidacja auth
6. Rozpoznawanie > przypominanie	4	Siatka + skróty boczne
7. Elastyczność i efektywność	4	Adaptacja skraca dostęp do częstych funkcji
8. Estetyka i minimalizm	4	Ukryto wagi i zbędne przełączniki
9. Pomoc w błędach	3	Komunikaty auth; narzędzia bez złożonych błędów
10. Pomoc i dokumentacja	2	README w repo; brak pomocy in-app

Tabela 5. Ewaluacja heurystyczna (średnia ≈ 3,4 / 4)

Wynik Q2: UI oceniono jako dobre; główny deficyt to brak wbudowanej pomocy kontekstowej (akceptowalne w skali prototypu zaliczeniowego).

11.5. Cognitive walkthrough i testy ręczne (Q3)

Wykonano przejście kognitywne dla 4 zadań. Kryterium sukcesu: zadanie wykonane bez pomocy w ≤3 minutach, bez błędów blokujących.

ID	Scenariusz	Kroki oczekiwane	Wynik
S1	Nowe konto + otwarcie kalkulatora	Rejestracja → siatka → Kalkulator	PASS
S2	Po 8 użyciach kalkulator w top sidebar	Powtarzane otwarcia → pojawienie w top 5	PASS
S3	Przypięcie Notatek, mimo niskiej wagi	Pin → Notatki na górze listy	PASS
S4	Drugie konto ma niezależny ranking	Wyloguj → nowe konto → domyślny układ	PASS
S5	Reset przywraca układ początkowy	Reset ×2 → kolejność alfabetyczna/domyślna	PASS

Tabela 6. Wyniki testów scenariuszowych

Wynik Q3: 5/5 scenariuszy zakończonych powodzeniem.

11.6. Ograniczenia badań

- Eksperyment A opiera się na danych syntetycznych — nie zastępuje badania z N≥12 uczestnikami.
- Ewaluacja heurystyczna wykonana przez autorów (ryzyko biasu).
- Brak pomiaru czasu zadania z eye-tracking / SUS na grupie zewnętrznej (rekomendowane jako dalsza praca).

12. Dyskusja

Uzyskane wyniki potwierdzają, że nawet prosty, interpretowalny model adaptacji (bez ML) może poprawić zgodność rankingu z aktualnymi preferencjami. Ukrycie wag w UI okazało się trafne z perspektywy minimalizmu — użytkownik doświadcza efektu (kolejność), a nie mechaniki. Konta lokalne skutecznie izolują personalizację, co jest kluczowe przy demonstracji na współdzielonym komputerze.

Kompromisem jest brak synchronizacji między urządzeniami oraz ograniczona ochrona danych względem atakującego z dostępem do przeglądarki. Dla celów dydaktycznych ICK jest to świadomie przyjęte ograniczenie zakresu.

13. Wnioski

1. Zrealizowano działającą aplikację SPA spełniającą wymagania tematu 17.
2. Wdrożono ważony ranking użyć (half-life 7 dni) oraz kontrolę użytkownika (pin, reset).
3. Wprowadzono multi-user: każde konto ma własny panel i dane narzędzi.
4. Badania wskazują przewagę modelu wagowego przy zmianie preferencji oraz dobrą jakość heurystyczną UI (śr. 3,4/4).
5. Możliwe rozszerzenia: synchronizacja chmurowa, predykcja kontekstowa (pora dnia), badanie SUS na grupie studentów, A/B test M1 vs M2 in-vivo.

14. Literatura

1. Nielsen J., *Heuristic evaluation*, w: Nielsen J., Mack R.L. (red.), Usability Inspection Methods, John Wiley & Sons, 1994.
2. Findlater L., McGrenere J., *A comparison of static, adaptive, and adaptable menus*, CHI 2004.
3. Gajos K.Z., Czerwinski M., Tan D.S., Weld D.S., *Exploring the design space for adaptive graphical user interfaces*, AVI 2006.
4. Ding Y., Li X., *Time weight collaborative filtering*, CIKM 2005.
5. Norman D.A., *The Design of Everyday Things*, Basic Books (wyd. poprawione).
6. Fitts P.M., *The information capacity of the human motor system…*, Journal of Experimental Psychology, 1954.
7. Shneiderman B. i in., *Designing the User Interface*, Pearson.
8. Dokumentacja React, Vite oraz Web Crypto API (MDN) — stan na 2026.

15. Załączniki

Załącznik A. Lista narzędzi

#	ID	Nazwa
1	notes	Notatki
2	calculator	Kalkulator
3	timer	Timer
4	converter	Konwerter jednostek
5	todos	Lista zadań
6	stopwatch	Stoper
7	password	Generator haseł
8	counter	Licznik
9	dice	Kostka
10	random	Losowa liczba
11	color	Selektor kolorów
12	countdown	Odliczanie daty
13	bmi	Kalkulator BMI
14	percent	Procenty
15	textcase	Zmiana wielkości liter
16	wordcount	Licznik znaków
17	tip	Napiwek
18	age	Wiek

Załącznik B. Instrukcja uruchomienia

```
git clone https://github.com/wpacocha/ick-adaptacyjny-panel.git
cd ick-adaptacyjny-panel
npm install
npm run dev
# aplikacja: http://localhost:5173/
```

Załącznik C. Mapowanie RF → implementacja

RF	Moduł
RF-01, RF-10	`useAuth.js`, `AuthScreen.jsx`
RF-02	`statsKeyForUser`, `toolKeyForUser`
RF-03	`data/tools.js` + `src/tools/*`
RF-04–RF-07	`useToolStats.js`, `App.jsx`, `Sidebar.jsx`
RF-08, RF-09	`togglePin`, `resetStats`

Załącznik D. Protokół testów ręcznych (skrót)

1. Wyczyść dane strony w DevTools → Application → Local Storage.
2. Zarejestruj użytkownika A, użyj 10× Kalkulator, 2× Timer.
3. Zweryfikuj, że Kalkulator jest w sidebarze.
4. Wyloguj, zarejestruj B — ranking pusty/domyślny.
5. Zaloguj ponownie A — stan zachowany.
6. Przypnij Timer, zresetuj historię — pin zostaje wg implementacji (reset czyści historię i piny do stanu domyślnego).